

本书各版全球总销量达1300万册
为新的C++11标准重新撰写

Stanley B. Lippman
Josée Lajoie 著
Barbara E. Moo

C++ (第5版) 中文版 Primer

王刚 杨巨峰 译

叶劲峰 李云 刘未鹏 审校
陈梓瀚 侯凤林



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
<http://www.phei.com.cn>

C++ Primer 第5版

顶级畅销的程序设计教程和参考书籍 为新的C++11标准重新撰写

本书针对最新发布的C++11标准进行了彻底的更新和修改，其中对C++语言权威而又全面的介绍将帮助你更快地学习这门编程语言，并且能以一种现代、高效的方式使用它。通过强调一些现代C++编程的最佳方法，作者展示了如何使用核心语言特性和标准库来编写高效、易读、强大的代码。

《C++ Primer（第5版）》从一开始就介绍C++标准库相关内容，利用标准库功能和设施来帮助你编写有用的程序，而不要求你首先掌握每个语言细节。书中很多例子都已经修订过，以使用新的语言特性，以及展示如何更好地利用它们。本书是C++新特性的值得信赖的教程，是C++核心概念和技术的权威介绍。对有经验的程序员，特别是那些迫切希望一探C++11增强特性究竟的程序员，本书也是很有价值的。

Stanley B. Lippman曾担任喷气推进实验室特别顾问，微软Visual C++开发组架构师，贝尔实验室技术部门成员，以及迪斯尼、梦工厂、皮克斯和PDI工作室的动画长片首席软件工程师。**Josée Lajoie**现供职于皮克斯，曾为IBM加拿大C/C++编译器开发团队成员，并曾担任最初的ANSI/ISO C++标准委员会核心语言特性工作组的主席。

Barbara E. Moo有近三十年的软件开发经验，她曾在AT&T工作了15年，其间与C++的发明人Bjarne Stroustrup密切合作，并曾数年担任C++开发团队的管理工作。

快速入门，收获更多

◎学习如何使用新的C++11的语言特性和标准库来快速构建健壮的程序，并感受高级语言程序设计的轻松自如。

◎通过一些展示当前最佳编码风格和程序设计技术的例子来学习。

◎理解“规则背后的原理”：为什么C++11是这样的。

◎使用大量的交叉引用来帮助你将有关联的概念和见解联系起来。

◎得益于新式的学习辅助和强调关键点的课后练习，帮助你避免一些陷阱，促进你在实践中养成好的习惯，以及巩固你所学的知识。

从informit.com/title/0321714113
获取扩展示例的源码

PEARSON

www.pearson.com



策划编辑：张春雨
责任编辑：刘 舫
封面设计：李 玲



程语言/C++

7-8-7-121-15535-2



9 787121 155352 >

定价：128.00元

Stanley B. Lippman
Josée Lajoie 著
Barbara E. Moo

(第5版)

C++ Primer

中文版

王刚 杨巨峰 译

叶劲峰 李云 刘未鹏
陈梓瀚 侯凤林 审校

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

内 容 简 介

这本久负盛名的C++经典教程,时隔八年之久,终于迎来史无前例的重大升级。除令全球无数程序员从中受益,甚至为之迷醉的——C++大师Stanley B. Lippman的丰富实践经验, C++标准委员会原负责人Josée Lajoie对C++标准的深入理解,以及C++先驱Barbara E. Moo在C++教学方面的真知灼见外,更是基于全新的C++11标准进行了全面而彻底的内容更新。非常难能可贵的是,书中所有示例均全部采用C++11标准改写,这在经典升级版中极其罕见——充分体现了C++语言的重大进展及其全面实践。书中丰富的教学辅助内容、醒目的知识点提示,以及精心组织的编程示范,让这本书在C++领域的权威地位更加不可动摇。无论是初学者入门,或是中高级程序员提升使用,本书均为不容置疑的首选。

Authorized translation from the English language edition, entitled C++ Primer, 5E, 9780321714114 by STANLEY B. LIPPMAN; JOSEE LAJOIE; BARBARA E. MOO, published by Pearson Education, Inc., publishing as Addison-Wesley Professional, Copyright©2013 Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD., and PUBLISHING HOUSE OF ELECTRONICS INDUSTRY Copyright ©2013

本书简体中文版专有出版权由Pearson Education培生教育出版亚洲有限公司授予电子工业出版社。未经出版者预先书面许可,不得以任何方式复制或抄袭本书的任何部分。

本书简体中文版贴有Pearson Education培生教育出版集团激光防伪标签,无标签者不得销售。

版权贸易合同登记号 图字: 01-2013-2487

图书在版编目(CIP)数据

C++ Primer中文版: 第5版 / (美)李普曼(Lippman,S.B.), (美)拉乔伊(Lajoie,J.), (美)默(Moo,B.E.)著; 王刚, 杨巨峰译. —北京: 电子工业出版社, 2013.9

书名原文: C++ Primer, 5E

ISBN 978-7-121-15535-2

I. ①C… II. ①李… ②拉… ③默… ④王… ⑤杨… III. ①C语言—程序设计 IV. ①TP312

中国版本图书馆CIP数据核字(2013)第169583号

策划编辑: 张春雨

责任编辑: 刘 舫

印 刷: 涿州市京南印刷厂

装 订: 涿州市京南印刷厂

出版发行: 电子工业出版社

北京市海淀区万寿路173信箱 邮编: 100036

开 本: 787×1092 1/16 印张: 54 字数: 1521千字

印 次: 2013年11月第2次印刷

印 数: 20001~30000册 定价: 128.00元

凡所购买电子工业出版社图书有缺损问题, 请向购买书店调换。若书店售缺, 请与本社发行部联系, 联系及邮购电话: (010) 88254888。

质量投诉请发邮件至zltts@phei.com.cn, 盗版侵权举报请发邮件至dbqq@phei.com.cn。

服务热线: (010) 88258888。

*To Beth,
who makes this,
and all things,
possible.*

*To Daniel and Anna,
who contain
virtually
all possibilities.*
—SBL

*To Mark and Mom,
for their
unconditional love and support.*
—JL

*To Andy,
who taught me
to program
and so much more.*
—BEM

推荐序 1

C++一直是我最为钟情的程序设计语言，我曾经在有些场合下提到“C++正在成为一门完美的程序设计语言”。从C++标准1998年版本到2011年版本的变化，基本上印证了我的这一提法。原来版本中来不及引入的内容，以及语言机制中发现的一些缺陷，都在新版本中得以弥补和发展。比如新版标准中引入了无序容器，以弥补原版标准中对hash容器的缺漏；新版标准支持移动构造函数和移动赋值运算符，以减小特定场景下对象拷贝的性能开销。新版标准不仅在语法上增加了大量特性，而且在标准库里也引入大量设施，使得标准库对于C++语言的重要性远超从前。

“完美的程序设计语言”，听起来很好，但代价是语言变得越来越复杂。从一个完善的类型系统或者一门程序设计语言的角度来看，新版本的C++标准是一个里程碑，但是，从C++学习者和使用者角度来看这未必是好事。语言的复杂性导致学习难度增加，学习周期变长；C++程序员写出好程序的门槛也相应提高。这差不多正是C++语言这几年的现状。我相信，随着计算机科学技术的发展，这种状况未来还会加剧。即便如此，我仍然乐于看到C++语言走向完美。

我与《C++ Primer》这本书的缘分从第3版开始，2001年有机会将其翻译成中文版本。当时，我使用C++已将近十年，通过这本书才第一次全面地梳理了实践中积累起来的C++知识。本书第3版是对1998版标准的全面诠释，我相信至今无出其右者。时隔12年以后，这本书第5版出版，虽然叙述风格跟第3版完全不同，但它在内容上全面顾及到2011版C++标准。第5版之于2011版标准，如同第3版之于1998版标准，必将成为经典的学习读本。

阅读这本书可以全面了解2011版本C++标准的内容。以三位作者在C++语言发展历程中的经历，本书的权威性自不容置疑：既有编译器的开发和实践，又参与C++标准的制定，再加上丰富的C++教学经历。如果说本书第3版是针对C++语言的特性和设计思想来展开讲述，那么，第5版则更加像一本学习教程，由浅入深，并结合大量代码实例来讲述C++语法和标准库。此外，由于本书的全面性，读者也可以将本书当作参考书，以备随时查阅。

本书在讲解的时候，常常会提到“编译器会如何如何”，学习语言的一个境界是把自己想象成编译器，这种要求对于一般的程序设计语言可能太高了，但是对于学习C和C++语言是最理想的方法。像编译器一样来思考和理解C++语言，如果暂时做不到，也不要紧，当有了一定的编写程序经验以后，在“揣摩”编译器行为的过程中可逐渐掌握C++语法特性。因此，本书值得阅读多遍，每多读一遍，就会加深理解。可能是考虑到篇幅的原因，本书很多地方没有展开来透彻地讲解。我相信，作者们已经在深度和广度上做了较为理想的折中。

本书的另一个特色是将C++的语法和标准库融为一体来介绍。C++标准库本身就是C++语法的最佳样例，其中包含不少C++高级特性的指导性用法。在我的程序设计经历中，有些C++语言特性（比如虚拟继承），我只在标准库中看到过实用做法。本书贯穿始终融合了C++标准库的知识和用法，这符合新版本C++标准的发展和变化，也符合现代软件开发现状。

最后，结合我在工程实践中使用和倡导 C++ 语言的经验，我想提一个关于学习和使用 C++ 语言的“两面性”观点。如前所述，C++ 语言正在走向完美，所以，C++ 语言值得学习（甚至研究），这些知识可以成为一切编程的基础。然而，在实践中，不必全面地使用 C++ 语言的各种特性，而应根据工程项目的实际情况，适当取舍（譬如动态类型信息、虚拟继承、异常等特性的使用很值得商榷）。通常只鼓励使用 C++ 语言的一个子集就够了，一个值得学习和参考的例子是 Google 发布的 Google C++ Style Guide。尽管在工程中只使用 C++ 的子集，但全面地学习 C++ 语言仍然是必要的，毕竟 C++ 语言是一个整体，并且 C++ 标准库自身全面地使用了 C++ 语言的各种特性。我自己在过去多年的实践中就一直恪守着这种两面的做法。

很幸运，我有机会在本书正式出版以前读到中文翻译版，译文通顺，术语规范。作为经典权威之作的最新版本，本书值得拥有。

潘爱民

2013 年 8 月于杭州

推荐序 2

C++11 标准公布之后，C++ 社群出现了久违的热情，有人甚至叫出“C++ 的复兴”。指望 C++ 回到 20 世纪 90 年代中期那样的地位显然是昧于大势的奢望，但是 C++ 经历了这么多年的打磨与起伏，其在工业界的地位已经非常稳固，在很多领域里已经是不可取代也没必要被取代的统治者。新标准的出现能够大大提升 C++ 开发的效率和质量，因此赢得欢呼也是情理之中。在这种氛围之下，编译器实现的速度也令人惊喜。短短两年时间，从开源的 GCC、LLVM 到专有的 Visual C++ 和 Intel C++，对于新标准的追踪之快，覆盖之全，与当年 C++98 标准颁布之后迟迟不能落地的窘境相比，可谓对比强烈。当年是热情的开发者反复敦促厂商实现完整标准而不得，为此沮丧无奈，那种心情，至今记忆犹新。时过境迁，今天是编译器实现远远冲在前面，开发者倒是大大地落在了后面。

时至今日，能够基本了解 C++11 标准的程序员恐怕不多，而能够以新的 C++ 风格开发实践的人更是凤毛麟角。因此，今天的 C++ 开发者面临的一个重要任务就是快速掌握新的 C++ 风格和工具。

而说到教授“正宗的”C++11 编程风格，《C++ Primer（第 5 版）》如同它之前的版本一样，扮演着法定教科书的角色。

一种优秀的编程语言，一定要对于计算这件事情实现一个完整和自洽的抽象。十几年来编程语言领域的竞争，除却实现质量之外，基本上是在比拼抽象的设计。而 C 语言之所以四十年长盛不衰，根本在于它对于现代计算机提供了一个底层的高级抽象：凡是比它低的抽象都过于简陋，凡是比它高的抽象都可以用 C 语言构造出来。C++ 成功的根本原因，恰恰是因为它虽然试图提供一些高级的抽象机制，但是其根基与 C 在同一层面。正因为如此，每当你需要走下去直接与硬件对话时，C++ 成为 C 之外唯一有效率的选择。我的一个朋友在进行了多年的大型系统软件开发之后，不无感慨地说，C++ 最大的力量不在于其抽象，恰恰在于其不抽象。

话虽然如此，但是 C++ 之所以脱离 C 而存在，毕竟还是因为其强大的抽象能力。Bjarne Stroustrup 曾经总结说，C++ 同时支持 4 种不同的编程风格：C 风格、基于对象、面向对象和泛型。事实上，把微软的 COM 也算进来的话，还可以加上一种“基于组件”的风格。这么多的风格共存于一种语言，就是其强大抽象机制的证明。但是，在 C++11 以前，C++ 的抽象可以说存在若干缺陷，其中最严重的是缺少自动内存管理和对象级别的消息发送机制。今天看来，C++98 只能说是特定历史条件造成的半成品，无论是从语言机制，还是标准库完备程度来说，可以说都存在明显的、不容忽略的缺陷。其直接后果，就是优雅性的缺失和效率的降低。我本人在十年前曾经与当时中国 C++ 社群中不少杰出的人物交流探讨，试图从 C++98 中剪裁出一个小巧、优雅的、自成一体的子集，希望至少在日常编程中，能够在这个子集之内可以写出与当时的 Java 和 C# 同样干净明晰的代码。为此我们尝试了各种古怪的模板技巧，并且到处寻找有启发的代码和经验来构造这个语言子集，结果并不理想，甚至可以说是令人非常失望。后来我在我的博客中发表过好几篇文章，探讨所谓的 C++ 风格问题，其实就是说，C++ 不支持简洁明快的面向对象风格，大家还不如回到基于对象甚至 C 语言的风格，最多加点模板，省一点代码量。非要面向对象的话，就必须依赖像 Qt 或者 MFC 那样的基础设施才可以。

C++11 出来之后，增强的语言机制和大为完善的标准库，为 C++ 语言的编程风格带来了革命性的变化。如果能够纯熟地运用 C++11 的新特征、新机制，那么就能够形成一种简洁优雅的 C++ 编程风格，以比以前更高的效率、更好的质量进行软件开发。对于这种新的风格，我认为“直觉、自然”是最佳的描述。也就是说，解决任何问题不必拘泥于什么笼罩一切的编程思想，也不再沉溺于各种古怪的模板技巧中无法自拔，而是能够根据那个问题本身采用最自然、最符合直觉的方式。C++ 有自己的一套思维方式，比如容器、算法、作为概念抽象的对象等，很大程度上这套思维方式确实是合乎直觉的。只有到了 C++11 这一代，C++ 语言的高级抽象才基本完备，这样一种风格才可能真正落实。因此可以说 C++11 对于 C++98 而言，不是一次简单的升级，而是一次本质的跃升。

学习新的 C++ 风格，并不是轻而易举的事情。即便对于以前已经精通 C++ 的人来说，熟练掌握 `rvalue reference`、`move` 语义，了解 `unique_ptr`、`shared_ptr` 和 `weak_ptr` 的完整用法，明智地使用 `function/bind` 和 `lambda` 机制，学习 C++ Concurrency 的新技术，都绝非一朝一夕之功。对于那些初学者来说，这件事情更不简单。

本书无论对于初学者还是提高者，都是最经典的教科书。一直以来，它的特点就是完整而详细，基本上关于语言本身的问题，都可以在这本书里得到解决。而本书的另一个重要优点，就是其完全基于新的编程风格编写，所有的例子和讲解都遵循 C++11 标准所体现出来的思路和风格进行，如果能够静下心来认真学习和练习，那么就能“一次到位”地掌握 C++11，尽管可能会比较慢。有经验的 C++ 开发者阅读这本书当然不用从头到尾，选择自己关心的内容学习 C++11 的新特性就可以，是快速提升自身能力的捷径。

差不多十年前，我提出一个观点，每一个具体的技术领域，只需要读四五本书就够了。以前的 C++ 是个例外，因为语言设计有缺陷，所以要读很多书才知道如何绕过缺陷。现在的 C++11 完全可以了，大家读四五本书就可以达到合格的水平，这恰恰是语言进步的体现。

本书是这四五本中的一本，而且是“教程+参考书”，扛梁之作，初学者的不二法门。另一本是《C++ 标准程序库》，对于 C++ 熟手来说更为快捷。Scott Meyers 的 *Effective C++* 永远是学习 C++ 者必读的，只不过这本书的第 4 版不知道什么时候出来。Anthony Williams 的 *C++ Concurrency in Action* 是学习用标准 C++ 开发并发程序的最佳选择。国内的作品，我则高度推荐陈硕的《Linux 多线程服务端编程》。这本书的名字赶跑了不少潜在的读者，所以我要特别说明一下。这本书是 C++ 开发的高水平作品，与其说是教你怎么用 C++ 写服务端开发，不如说是教你如何以服务端开发为例子提升 C++ 开发水平。前面几本书都是谈标准 C++ 自己的事情，碰到像 `iostream` 这样失败的标准组件也不得不硬着头皮介绍。而这本书是接地气的实践结晶，告诉你面对具体问题时应怎样权衡，C++ 里什么好用，什么不好用，为什么，等等。

今天的 C++ 学习者是非常幸运的，可以在 C++11 这个基础上大步向前，不必再因为那些语言的缺陷和过度的技巧而烦恼。大家静下心来认真读几本书，可以打下很好的基础。

孟岩

2013 年 8 月于北京

前言

难以计数的程序员已经通过旧版的《C++ Primer》学会了 C++ 语言。而在这段时间中，C++ 本身又已成熟了许多：语言本身的关注点和程序设计社区的关注点都已大大开阔，已经从主要关注机器效率转变为更多地关注编程效率。

2011 年，C++ 标准委员会发布了 ISO C++ 标准的一个重要修订版。此修订版是 C++ 进化过程中的最新一步，延续了前几个版本对编程效率的强调。新标准的主要目标是：

- 使语言更为统一，更易于教学。
- 使标准库更简单、安全、使用更高效。
- 使编写高效率的抽象和库变得更简单。

因此，在这个版本的《C++ Primer》中，我们进行了彻底的修改，使用了最新的 C++ 标准，即 C++11。为了了解新标准是如何全面影响 C++ 语言的，你可以看一下 XXIII 页至 XXV 页的新特性列表，其中列出了哪些章节涉及了 C++ 的新特性。

新标准增加的一些特性是具有普适性的，例如用于类型推断的 `auto`。这些新特性使本书中的代码更易于阅读和理解。程序（以及程序员！）可以忽略类型的细节，从而更容易集中精力于程序逻辑上来。其他一些新特性，例如智能指针和允许移动的容器，允许我们编写更为复杂的类，而又不必与错综复杂的资源管理做斗争。因此，在本书中开始讲授如何编写自己的类，会比第 4 版简单得多。旧标准中阻挡在我们前进路上的很多细节，你都不必再担心了。

对于本书中涉及新标准定义的新特性的那些部分，我们都已用一个特殊的图标标记出来了。我们希望这些提示标记对那些已经熟悉 C++ 语言核心内容的读者是有帮助的，可以帮助他们决定将注意力投向哪里。对于那些可能尚不支持所有新特性的编译器，我们还希望这些图标能有助于解释这类编译器所给出的编译错误信息。这是因为虽然本书中几乎所有例子都已经用最新版本的 GNU 编译器编译通过，但我们知道一些读者可能尚未将编译器更新到最新版本。虽然新标准增加了大量新功能，但核心 C++ 语言并未变化，这构成了本书的大部分内容。读者可以借助这些图标来判断哪些功能可能还没有被自己的编译器所支持。

C++
11

为什么选择这本书？

现代 C++ 语言可以看作是三部分组成的：

- 低级语言，大部分继承自 C 语言。
- 现代高级语言特性，允许我们定义自己的类型以及组织大规模程序和系统。
- 标准库，它利用高级特性来提供有用的数据结构和算法。

大多数 C++ 教材按照语言进化的顺序来组织其内容。首先讲授 C++ 的 C 子集，然后将 C++ 中更为抽象的一些特性作为高级话题在书的最后进行介绍。这种方式存在两个问题：读者会陷入那些继承自低级程序设计的细节，从而由于挫折感而放弃；读者被强加学习一些坏习惯，随后又需要忘记这些内容。

我们采用一种相反的方法：从一开始就介绍一些语言特性，能让程序员忽略那些继承自低级程序设计的细节。例如，在介绍和使用内置的算术和数组类型时，我们还连同介绍和使用标准库中的类型 `string` 和 `vector`。使用这些类型的程序更易写、易理解且更少出错。

太多时候，标准库被当作一种“高级”话题来讲授。很多教材不使用标准库，而是使用基于字符数组指针和动态内存管理的低级程序设计技术。让使用这种低级技术的程序正确运行，要比编写相应的使用标准库的 C++ 代码困难得多。

贯穿全书，我们都在强调好的风格：我们想帮助读者直接养成好的习惯，而不是在获得很多很复杂的知识后再去忘掉那些坏习惯。我们特别强调那些棘手的问题，并对常见的错误想法和陷阱提出警告。

我们还注意解释规则背后的基本原理——使读者不仅知其然，还能知其所以然。我们相信，通过体会程序的工作原理，读者会更快地巩固对语言的理解。

虽然你不必为了学习本书而掌握 C 语言，但我们还是假定你了解足够多的程序设计知识，了解至少一门现代分程序结构语言，知道如何用这门语言编写、编译以及运行程序。特别是，我们假定你已经使用过变量，编写、调用过函数，也使用过编译器。

第 5 版变化的内容

这一版《C++ Primer》的新特点是用边栏图标来帮助引导读者。C++ 是一种庞大的编程语言，它提供了一些为特定程序设计问题定制的功能。其中一些功能对大型项目团队有很重要的意义，但对于小型项目开发可能并无必要。因此，并非每个程序员都需要了解每种语言特性的所有细节。我们加入这些边栏图标来帮助读者弄清哪些内容可以随后再学习，而哪些主题是更为重要的。



对于包含 C++ 语言基础内容的章节，我们用一个小人正在读书的图标加以标记。用这个图标标记的那些章节，涵盖了构成语言核心部分的主题。每个人都应该阅读并理解这些章节的内容。



对于那些涉及高级主题或特殊目的主题的章节，我们也进行了标记。在首次阅读时，这些章节可以跳过或快速浏览。我们用一叠书的图标标记这些章节，指出在这些地方，你可以放心地放下书本。快速浏览这些章节可能是一个好主意，这样你就可以知道有这些特性存在。但在真正需要自己的程序中使用这些特性之前，没有必要花费时间仔细学习这些主题。



为了进一步引导读者的注意力，我们还用放大镜图标标记了特别复杂的概念。我们希望读者对有这种标记的章节能多花费一些时间彻底理解其中的内容。在这些章节中，至少有一些，其主题的重要性可能不是那么明显；但我们认为，你会发现这些章节涉及的主题对理解 C++ 语言原来至关重要。

交叉引用的广泛使用，是本书采用的另外一种阅读帮助。我们希望这些引用能帮助读者容易地翻阅书中的内容，同时还能在后面的例子涉及到前面的内容时容易地跳回到前面。

没有改变的是，《C++ Primer》仍是一本清晰、正确、全面的 C++ 入门教材。我们通过给出一系列复杂度逐步增加的例子来讲授这门语言，这些例子说明了语言特性，展示了如何充分用好 C++ 语言。

本书的结构

我们首先在第 I 部分和第 II 部分中介绍了 C++ 语言和标准库的基础内容。这两部分包含的内容足够你编写出有意义的程序，而不是只能写一些玩具程序。大部分程序员基本上都需要掌握本书这两部分所包含的所有内容。

除了讲授 C++ 的基础内容，第 I 部分和第 II 部分还有另外一个重要目的：通过使用标准库中定义的抽象设施，使你更加适应高级程序设计技术。标准库设施本身是一组抽象数据类型，通常用 C++ 编写。用来设计标准库的，就是任何 C++ 程序员都可以使用的用来构造类的那些语言特性。我们讲授 C++ 语言的一个经验是，在先学习了使用设计良好的抽象类型后，读者会发现理解如何构造自己的类型更容易了。

只有在经过全面的标准库使用训练，并编写了各种标准库所支持的抽象程序后，我们才真正进入到那些允许你编写自己的抽象类型的 C++ 特性中去。本书的第 III 部分和第 IV 部分介绍了如何编写类的形式的抽象类型。第 III 部分包含基础内容，第 IV 部分介绍更专门的语言特性。

在第 III 部分中，我们将介绍拷贝控制问题，以及其他一些使类能像内置类型一样容易使用的技术。类是面向对象编程和泛型编程的基础，第 III 部分也会介绍这些内容。第 IV 部分是《C++ Primer》的结束部分，它介绍了一些在组织大型复杂系统时非常有用的语言特性。此外，我们将在附录 A 中总结标准库算法。

读者帮助

本书的每一章均以一个总结和一个术语表结束，两者一起扼要回顾了这一章的大部分学习重点。读者应该将这些部分作为个人备忘录：如果你不理解某个术语，可以重新学习这一章的相应部分。

在本书中我们还使用了其他一些学习辅助：

- 重要的术语用**黑体**显示，我们假定读者已经熟悉的重要术语用楷体显示。每个术语都会列在章末尾的术语表中。
- 贯穿全书，我们用灰底衬托的方式来提醒读者需要注意的重要部分，对常见的陷阱提出警告，建议好的程序设计习惯，以及提供一般性的使用提示。
- 为了更好地理解语言特性间和概念间的联系，我们提供大量向前的和向后的交叉引用。
- 对重要的概念和 C++ 新程序员常常觉得困难的主题，我们提供边栏讨论。
- 学习任何程序设计语言都需要编写程序。为此，贯穿全书我们提供大量程序示例。扩展示例的源码可从下面的网址获得：

<http://www.informit.com/title/0321714113>

- 正文中切口处以“”形式标注的页码为英文原版书中的页码，便于读者与英文原版书进行对照阅读。

关于编译器的注意事项

在撰写本书时（2012 年 7 月），编译器提供商正在努力工作，升级编译器以匹配最新的 ISO 标准。我们使用最多的编译器是 GNU 编译器 4.7.0。本书中只有一小部分特性在此编译器中尚未实现：继承构造函数、成员函数的引用限定符以及正则表达式库。

致谢

我们要特别感谢标准委员会几位现任和前任委员：Dave Abrahams、Andy Koenig、Stephan T. Lavavej、Jason Merrill、John Spicer 和 Herb Sutter 在准备本书的过程中提供的帮助。在理解新标准的一些更微妙之处，他们为我们提供了宝贵的帮助。我们还要感谢很多致力于升级 GNU 编译器以实现新标准的人们。

与旧版《C++ Primer》中一样，我们要感谢 Bjarne Stroustrup 不知疲倦地为 C++ 工作以及他和作者长时间的友谊。我们还要感谢 Alex Stepanov 的非凡洞察力，催生了标准库核心的容器和算法。最后，我们要感谢 C++ 标准委员会的所有委员，感谢他们这么多年来在净化、精炼和改进 C++ 语言方面的辛苦工作。

我们衷心感谢审稿人：Marshall Clow、Jon Kalb、Nevin Liber、Dr. C. L. Tondo、Daveed Vandevoorde 和 Steve Vinoski，他们建设性的意见帮助我们对全书做出了大大小小的改进。

最后，我们要感谢 Addison-Wesley 公司的优秀员工，他们指导了本书的整个出版过程：Peter Gordon，我们的编辑，他给了我们动力再次修改 *C++ Primer*；Kim Boedigheimer，保证了一切按计划进行；Barbara Wood，她在编辑过程中找到了大量编辑错误；还有 Elizabeth Ryan，很高兴再次和她共同工作，她指导我们完成了整个设计和生产流程。

目录

第 1 章 开始	1
1.1 编写一个简单的 C++ 程序	2
1.1.1 编译、运行程序	3
1.2 初识输入输出	5
1.3 注释简介	8
1.4 控制流	10
1.4.1 while 语句	10
1.4.2 for 语句	11
1.4.3 读取数量不定的输入数据	13
1.4.4 if 语句	15
1.5 类简介	17
1.5.1 Sales_item 类	17
1.5.2 初识成员函数	20
1.6 书店程序	21
小结	23
术语表	23
第 I 部分 C++ 基础	27
第 2 章 变量和基本类型	29
2.1 基本内置类型	30
2.1.1 算术类型	30
2.1.2 类型转换	32
2.1.3 字面值常量	35
2.2 变量	38
2.2.1 变量定义	38
2.2.2 变量声明和定义的关系	41
2.2.3 标识符	42
2.2.4 名字的作用域	43
2.3 复合类型	45
2.3.1 引用	45
2.3.2 指针	47
2.3.3 理解复合类型的声明	51
2.4 const 限定符	53
2.4.1 const 的引用	54
2.4.2 指针和 const	56
2.4.3 顶层 const	57
2.4.4 constexpr 和常量表达式	58

2.5 处理类型	60
2.5.1 类型别名	60
2.5.2 auto 类型说明符	61
2.5.3 decltype 类型指示符	62
2.6 自定义数据结构	64
2.6.1 定义 Sales_data 类型	64
2.6.2 使用 Sales_data 类	66
2.6.3 编写自己的头文件	67
小结	69
术语表	69
第 3 章 字符串、向量和数组	73
3.1 命名空间的 using 声明	74
3.2 标准库类型 string	75
3.2.1 定义和初始化 string 对象	76
3.2.2 string 对象上的操作	77
3.2.3 处理 string 对象中的字符	81
3.3 标准库类型 vector	86
3.3.1 定义和初始化 vector 对象	87
3.3.2 向 vector 对象中添加元素	90
3.3.3 其他 vector 操作	91
3.4 迭代器介绍	95
3.4.1 使用迭代器	95
3.4.2 迭代器运算	99
3.5 数组	101
3.5.1 定义和初始化内置数组	101
3.5.2 访问数组元素	103
3.5.3 指针和数组	105
3.5.4 C 风格字符串	109
3.5.5 与旧代码的接口	111
3.6 多维数组	112
小结	117
术语表	117
第 4 章 表达式	119
4.1 基础	120
4.1.1 基本概念	120
4.1.2 优先级与结合律	121
4.1.3 求值顺序	123
4.2 算术运算符	124
4.3 逻辑和关系运算符	126
4.4 赋值运算符	129
4.5 递增和递减运算符	131
4.6 成员访问运算符	133
4.7 条件运算符	134
4.8 位运算符	135

4.9	sizeof 运算符	139
4.10	逗号运算符	140
4.11	类型转换	141
4.11.1	算术转换	142
4.11.2	其他隐式类型转换	143
4.11.3	显式转换	144
4.12	运算符优先级表	147
小结		149
术语表		149
第 5 章	语句	153
5.1	简单语句	154
5.2	语句作用域	155
5.3	条件语句	156
5.3.1	if 语句	156
5.3.2	switch 语句	159
5.4	迭代语句	165
5.4.1	while 语句	165
5.4.2	传统的 for 语句	166
5.4.3	范围 for 语句	168
5.4.4	do while 语句	169
5.5	跳转语句	170
5.5.1	break 语句	170
5.5.2	continue 语句	171
5.5.3	goto 语句	172
5.6	try 语句块和异常处理	172
5.6.1	throw 表达式	173
5.6.2	try 语句块	174
5.6.3	标准异常	176
小结		178
术语表		178
第 6 章	函数	181
6.1	函数基础	182
6.1.1	局部对象	184
6.1.2	函数声明	186
6.1.3	分离式编译	186
6.2	参数传递	187
6.2.1	传值参数	187
6.2.2	传引用参数	188
6.2.3	const 形参和实参	190
6.2.4	数组形参	193
6.2.5	main: 处理命令行选项	196
6.2.6	含有可变形参的函数	197
6.3	返回类型和 return 语句	199
6.3.1	无返回值函数	200

6.3.2	有返回值函数	200
6.3.3	返回数组指针	205
6.4	函数重载	206
6.4.1	重载与作用域	210
6.5	特殊用途语言特性	211
6.5.1	默认实参	211
6.5.2	内联函数和 <code>constexpr</code> 函数	213
6.5.3	调试帮助	215
6.6	函数匹配	217
6.6.1	实参类型转换	219
6.7	函数指针	221
	小结	225
	术语表	225
第 7 章	类	227
7.1	定义抽象数据类型	228
7.1.1	设计 <code>Sales_data</code> 类	228
7.1.2	定义改进的 <code>Sales_data</code> 类	230
7.1.3	定义类相关的非成员函数	234
7.1.4	构造函数	235
7.1.5	拷贝、赋值和析构	239
7.2	访问控制与封装	240
7.2.1	友元	241
7.3	类的其他特性	243
7.3.1	类成员再探	243
7.3.2	返回 <code>*this</code> 的成员函数	246
7.3.3	类类型	249
7.3.4	友元再探	250
7.4	类的作用域	253
7.4.1	名字查找与类的作用域	254
7.5	构造函数再探	257
7.5.1	构造函数初始值列表	258
7.5.2	委托构造函数	261
7.5.3	默认构造函数的作用	262
7.5.4	隐式的类类型转换	263
7.5.5	聚合类	266
7.5.6	字面值常量类	267
7.6	类的静态成员	268
	小结	273
	术语表	273
第 II 部分	C++标准库	275
第 8 章	IO 库	277
8.1	IO 类	278
8.1.1	IO 对象无拷贝或赋值	279
8.1.2	条件状态	279

8.1.3 管理输出缓冲	281
8.2 文件输入输出	283
8.2.1 使用文件流对象	284
8.2.2 文件模式	286
8.3 string 流	287
8.3.1 使用 <code>istringstream</code>	287
8.3.2 使用 <code>ostringstream</code>	289
小结	290
术语表	290
第 9 章 顺序容器	291
9.1 顺序容器概述	292
9.2 容器库概览	294
9.2.1 迭代器	296
9.2.2 容器类型成员	297
9.2.3 <code>begin</code> 和 <code>end</code> 成员	298
9.2.4 容器定义和初始化	299
9.2.5 赋值和 <code>swap</code>	302
9.2.6 容器大小操作	304
9.2.7 关系运算符	304
9.3 顺序容器操作	305
9.3.1 向顺序容器添加元素	305
9.3.2 访问元素	309
9.3.3 删除元素	311
9.3.4 特殊的 <code>forward_list</code> 操作	312
9.3.5 改变容器大小	314
9.3.6 容器操作可能使迭代器失效	315
9.4 <code>vector</code> 对象是如何增长的	317
9.5 额外的 <code>string</code> 操作	320
9.5.1 构造 <code>string</code> 的其他方法	321
9.5.2 改变 <code>string</code> 的其他方法	322
9.5.3 <code>string</code> 搜索操作	325
9.5.4 <code>compare</code> 函数	327
9.5.5 数值转换	327
9.6 容器适配器	329
小结	332
术语表	332
第 10 章 泛型算法	335
10.1 概述	336
10.2 初识泛型算法	338
10.2.1 只读算法	338
10.2.2 写容器元素的算法	339
10.2.3 重排容器元素的算法	342
10.3 定制操作	344
10.3.1 向算法传递函数	344

10.3.2	lambda 表达式	345
10.3.3	lambda 捕获和返回	349
10.3.4	参数绑定	354
10.4	再探迭代器	357
10.4.1	插入迭代器	358
10.4.2	iostream 迭代器	359
10.4.3	反向迭代器	363
10.5	泛型算法结构	365
10.5.1	5 类迭代器	365
10.5.2	算法形参模式	367
10.5.3	算法命名规范	368
10.6	特定容器算法	369
小结		371
术语表		371
第 11 章	关联容器	373
11.1	使用关联容器	374
11.2	关联容器概述	376
11.2.1	定义关联容器	376
11.2.2	关键字类型的要求	378
11.2.3	pair 类型	379
11.3	关联容器操作	381
11.3.1	关联容器迭代器	382
11.3.2	添加元素	383
11.3.3	删除元素	386
11.3.4	map 的下标操作	387
11.3.5	访问元素	388
11.3.6	一个单词转换的 map	391
11.4	无序容器	394
小结		397
术语表		397
第 12 章	动态内存	399
12.1	动态内存与智能指针	400
12.1.1	shared_ptr 类	400
12.1.2	直接管理内存	407
12.1.3	shared_ptr 和 new 结合使用	412
12.1.4	智能指针和异常	415
12.1.5	unique_ptr	417
12.1.6	weak_ptr	420
12.2	动态数组	423
12.2.1	new 和数组	423
12.2.2	allocator 类	427
12.3	使用标准库：文本查询程序	430
12.3.1	文本查询程序设计	430
12.3.2	文本查询程序类的定义	432

小结	436
术语表	436
第III部分 类设计者的工具	437
第 13 章 拷贝控制	439
13.1 拷贝、赋值与销毁	440
13.1.1 拷贝构造函数	440
13.1.2 拷贝赋值运算符	443
13.1.3 析构函数	444
13.1.4 三/五法则	447
13.1.5 使用=default	449
13.1.6 阻止拷贝	449
13.2 拷贝控制和资源管理	452
13.2.1 行为像值的类	453
13.2.2 定义行为像指针的类	455
13.3 交换操作	457
13.4 拷贝控制示例	460
13.5 动态内存管理类	464
13.6 对象移动	470
13.6.1 右值引用	471
13.6.2 移动构造函数和移动赋值运算符	473
13.6.3 右值引用和成员函数	481
小结	486
术语表	486
第 14 章 重载运算与类型转换	489
14.1 基本概念	490
14.2 输入和输出运算符	494
14.2.1 重载输出运算符<<	494
14.2.2 重载输入运算符>>	495
14.3 算术和关系运算符	497
14.3.1 相等运算符	497
14.3.2 关系运算符	498
14.4 赋值运算符	499
14.5 下标运算符	501
14.6 递增和递减运算符	502
14.7 成员访问运算符	504
14.8 函数调用运算符	506
14.8.1 lambda 是函数对象	507
14.8.2 标准库定义的函数对象	509
14.8.3 可调用对象与 function	511
14.9 重载、类型转换与运算符	514
14.9.1 类型转换运算符	514
14.9.2 避免有二义性的类型转换	517
14.9.3 函数匹配与重载运算符	521
小结	523
术语表	523

第 15 章 面向对象程序设计	525
15.1 OOP: 概述	526
15.2 定义基类和派生类	527
15.2.1 定义基类	528
15.2.2 定义派生类	529
15.2.3 类型转换与继承	534
15.3 虚函数	536
15.4 抽象基类	540
15.5 访问控制与继承	542
15.6 继承中的类作用域	547
15.7 构造函数与拷贝控制	551
15.7.1 虚析构造函数	552
15.7.2 合成拷贝控制与继承	552
15.7.3 派生类的拷贝控制成员	554
15.7.4 继承的构造函数	557
15.8 容器与继承	558
15.8.1 编写 Basket 类	559
15.9 文本查询程序再探	562
15.9.1 面向对象的解决方案	563
15.9.2 Query_base 类和 Query 类	567
15.9.3 派生类	568
15.9.4 eval 函数	571
小结	575
术语表	575
第 16 章 模板与泛型编程	577
16.1 定义模板	578
16.1.1 函数模板	578
16.1.2 类模板	583
16.1.3 模板参数	592
16.1.4 成员模板	595
16.1.5 控制实例化	597
16.1.6 效率与灵活性	599
16.2 模板实参推断	600
16.2.1 类型转换与模板类型参数	601
16.2.2 函数模板显式实参	603
16.2.3 尾置返回类型与类型转换	604
16.2.4 函数指针和实参推断	607
16.2.5 模板实参推断和引用	608
16.2.6 理解 std::move	610
16.2.7 转发	612
16.3 重载与模板	614
16.4 可变参数模板	618
16.4.1 编写可变参数函数模板	620
16.4.2 包扩展	621
16.4.3 转发参数包	622

16.5 模板特例化	624
小结	630
术语表	630
第IV部分 高级主题	633
第 17 章 标准库特殊设施	635
17.1 tuple 类型	636
17.1.1 定义和初始化 tuple	637
17.1.2 使用 tuple 返回多个值	638
17.2 bitset 类型	640
17.2.1 定义和初始化 bitset	641
17.2.2 bitset 操作	643
17.3 正则表达式	645
17.3.1 使用正则表达式库	646
17.3.2 匹配与 Regex 迭代器类型	650
17.3.3 使用子表达式	653
17.3.4 使用 regex_replace	657
17.4 随机数	659
17.4.1 随机数引擎和分布	660
17.4.2 其他随机数分布	663
17.5 IO 库再探	666
17.5.1 格式化输入与输出	666
17.5.2 未格式化的输入/输出操作	673
17.5.3 流随机访问	676
小结	680
术语表	680
第 18 章 用于大型程序的工具	683
18.1 异常处理	684
18.1.1 抛出异常	684
18.1.2 捕获异常	687
18.1.3 函数 try 语句块与构造函数	689
18.1.4 noexcept 异常说明	690
18.1.5 异常类层次	693
18.2 命名空间	695
18.2.1 命名空间定义	695
18.2.2 使用命名空间成员	701
18.2.3 类、命名空间与作用域	705
18.2.4 重载与命名空间	708
18.3 多重继承与虚继承	710
18.3.1 多重继承	711
18.3.2 类型转换与多个基类	713
18.3.3 多重继承下的类作用域	715
18.3.4 虚继承	717
18.3.5 构造函数与虚继承	720
小结	722
术语表	722

第 19 章 特殊工具与技术	725
19.1 控制内存分配	726
19.1.1 重载 new 和 delete	726
19.1.2 定位 new 表达式	729
19.2 运行时类型识别	730
19.2.1 dynamic_cast 运算符	730
19.2.2 typeid 运算符	732
19.2.3 使用 RTTI	733
19.2.4 type_info 类	735
19.3 枚举类型	736
19.4 类成员指针	739
19.4.1 数据成员指针	740
19.4.2 成员函数指针	741
19.4.3 将成员函数用作可调用对象	744
19.5 嵌套类	746
19.6 union: 一种节省空间的类	749
19.7 局部类	754
19.8 固有的不可移植的特性	755
19.8.1 位域	756
19.8.2 volatile 限定符	757
19.8.3 链接指示: extern "C"	758
小结	762
术语表	762
附录 A 标准库	765
A.1 标准库名字和头文件	766
A.2 算法概览	770
A.2.1 查找对象的算法	771
A.2.2 其他只读算法	772
A.2.3 二分搜索算法	772
A.2.4 写容器元素的算法	773
A.2.5 划分与排序算法	775
A.2.6 通用重排操作	776
A.2.7 排列算法	778
A.2.8 有序序列的集合算法	778
A.2.9 最小值和最大值	779
A.2.10 数值算法	780
A.3 随机数	781
A.3.1 随机数分布	781
A.3.2 随机数引擎	783
索引	785

C++11 的新特性

2.1.1	long long 类型	31
2.2.1	列表初始化	39
2.3.2	nullptr 常量	48
2.4.4	constexpr 变量	59
2.5.1	类型别名声明	60
2.5.2	auto 类型指示符	61
2.5.3	decltype 类型指示符	62
2.6.1	类内初始化	65
3.2.2	使用 auto 或 decltype 缩写类型	79
3.2.3	范围 for 语句	82
3.3	定义 vector 对象的 vector (向量的向量)	87
3.3.1	vector 对象的列表初始化	88
3.4.1	容器的 cbegin 和 cend 函数	98
3.5.3	标准库 begin 和 end 函数	106
3.6	使用 auto 和 decltype 简化声明	115
4.2	除法的舍入规则	125
4.4	用大括号包围的值列表赋值	129
4.9	将 sizeof 用于类成员	139
5.4.3	范围 for 语句	168
6.2.6	标准库 initializer_list 类	197
6.3.2	列表初始化返回值	203
6.3.3	定义尾置返回类型	206
6.3.3	使用 decltype 简化返回类型定义	206
6.5.2	constexpr 函数	214
7.1.4	使用=default 生成默认构造函数	237
7.3.1	类对象成员的类内初始化	246
7.5.2	委托构造函数	261
7.5.6	constexpr 构造函数	268
8.2.1	用 string 对象处理文件名	284
9.1	array 和 forward_list 容器	293
9.2.3	容器的 cbegin 和 cend 函数	298
9.2.4	容器的列表初始化	300
9.2.5	容器的非成员函数 swap	303
9.3.1	容器 insert 成员的返回类型	308
9.3.1	容器的 emplace 成员	308
9.4	shrink_to_fit	318
9.5.5	string 的数值转换函数	327
10.3.2	lambda 表达式	346

10.3.3	lambda 表达式中的尾置返回类型	353
10.3.4	标准库 bind 函数	354
11.2.1	关联容器的列表初始化	377
11.2.3	列表初始化 pair 的返回类型	380
11.3.2	pair 的列表初始化	384
11.4	无序容器	394
12.1	智能指针	400
12.1.1	shared_ptr 类	400
12.1.2	动态分配对象的列表初始化	407
12.1.2	auto 和动态分配	408
12.1.5	unique_ptr 类	417
12.1.6	weak_ptr 类	420
12.2.1	范围 for 语句不能应用于动态分配数组	424
12.2.1	动态分配数组的列表初始化	424
12.2.1	auto 不能用于分配数组	424
12.2.2	allocator::construct 可使用任意构造函数	428
13.1.5	将=default 用于拷贝控制成员	449
13.1.6	使用=delete 阻止拷贝类对象	449
13.5	用移动类对象代替拷贝类对象	469
13.6.1	右值引用	471
13.6.1	标准库 move 函数	472
13.6.2	移动构造函数和移动赋值	473
13.6.2	移动构造函数通常应该是 noexcept	473
13.6.2	移动迭代器	480
13.6.3	引用限定成员函数	483
14.8.3	function 类模板	512
14.9.1	explicit 类型转换运算符	516
15.2.2	虚函数的 override 指示符	530
15.2.2	通过定义类为 final 来阻止继承	533
15.3	虚函数的 override 和 final 指示符	538
15.7.2	删除的拷贝控制和继承	553
15.7.4	继承的构造函数	557
16.1.2	声明模板类型形参为友元	590
16.1.2	模板类型别名	590
16.1.3	模板函数的默认模板参数	594
16.1.5	实例化的显式控制	597
16.2.3	模板函数与尾置返回类型	605
16.2.5	引用折叠规则	609
16.2.6	用 static_cast 将左值转换为右值	612
16.2.7	标准库 forward 函数	614
16.4	可变参数模板	618
16.4	sizeof... 运算符	619
16.4.3	可变参数模板与转发	622
17.1	标准库 tuple 类模板	636

17.2.2	新的 <code>bitset</code> 运算	643
17.3	正则表达式库	645
17.4	随机数库	659
17.5.1	浮点数格式控制	670
18.1.4	<code>noexcept</code> 异常指示符	690
18.1.4	<code>noexcept</code> 运算符	691
18.2.1	内联命名空间	699
18.3.1	继承的构造函数与多重继承	712
19.3	有作用域的 <code>enum</code>	736
19.3	说明类型用于保存 <code>enum</code> 对象	738
19.3	<code>enum</code> 的前置声明	738
19.4.3	标准库 <code>mem_fn</code> 类模板	746
19.6	类类型的 <code>union</code> 成员	751